Monocular Depth Estimation

Monocular Depth Estimation

1. Course Content
2. Case Demonstration
3. Core Source Code Analysis

1. Course Content

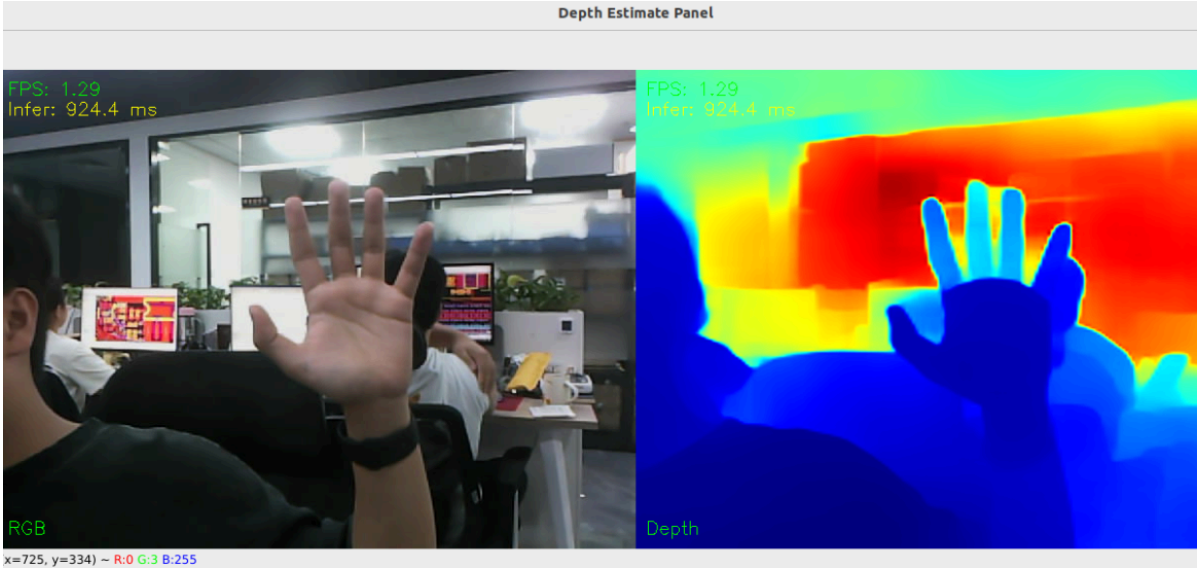
- Use the yolo26 depth estimation model to estimate in real time the physical depth distance of each region and object in the scene relative to the camera from an RGB color image, and map the relative depth to an intuitive pseudo-color heatmap output.

> **Important**
>
> The Raspberry Pi main board has no GPU or BPU to accelerate model inference, so the frame rate may be slightly lower at runtime.

2. Case Demonstration

```
ros2 launch yahboomcar_bringup camera_demo.launch.py demo:=depth_estimate
```

- **Pops up a dual-frame real-time window**: A window titled `Depth Estimate Panel` is shown, with the original RGB color frame on the left and the monocular depth estimation result on the right.




```
Frame Display Description:
- Left frame 【RGB】: real-time color camera frame, with the current frame rate (FPS) and single-frame inference time (Infer ms) shown in the upper left corner;
- Right frame 【Depth】: the dense relative depth pseudo-color map generated by monocular estimation (default JET mode; warm colors/red indicate nearby objects, cold colors/blue indicate distant background).
```

3. Core Source Code Analysis

The source file is located at:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_depth/yahboomcar_depth/Yolov26/depth_estimate.py
```

_init_arm_pose function description:

- `wait_for_service(timeout_sec=10.0)`: Waits for the robot arm joint control service to come online; if it times out after 10 seconds, the initial pose setup is skipped.
- `ArmPosition.Request()`: Constructs a joint control request, where `request_joints` is the initial pose angle list.
- `call_async(req)`: Sends the joint control request asynchronously.
- `add_done_callback(self._init_arm_done_callback)`: Registers a callback function to check the execution result after the request completes.

_draw_info function description:

- A helper function that overlays and draws the FPS (green) and inference time (yellow) in the upper left corner of the image.

get_RGBImageCallBack function description:

- `imgmsg_to_cv2(rgb_msg, desired_encoding='bgr8')`: Converts the ROS Image message into an OpenCV BGR format image.
- `self.depth_model.predict(rgb_image, stream=True, verbose=False)`: Performs depth estimation inference on the current frame; `stream=True` enables streaming output to save memory.
- `getattr(res, 'depth', None)`: Extracts the depth map data from the result object; returns None if it does not exist.
- `depth_map.data`: If the depth map is a wrapper object, extracts the underlying data.
- `depth_map.cpu().numpy()`: If the depth map is on the GPU, it is first moved to the CPU and then converted to a NumPy array.
- `np.squeeze(depth_map)`: Removes redundant axes of size 1 in the array, ensuring it is a 2D matrix.
- `cv2.normalize(depth_map, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)`: Normalizes the depth values to the 0~255 range and converts them to the uint8 type, facilitating color mapping.
- `cv2.applyColorMap(depth_vis, self.colormap)`: Applies the pseudo-color mapping, converting the grayscale depth map into a color heatmap.
- `cv2.resize(depth_vis, ...)`: If the depth map size is inconsistent with the RGB image, it is resized to the same size for concatenated display.
- FPS calculation: Counts the number of frames within 1 second to calculate the real-time frame rate.
- `cv2.putText(rgb_image, "RGB", ...)` / `cv2.putText(depth_vis, "Depth", ...)`: Labels the left and right frames with the "RGB" and "Depth" tags at the bottom respectively.

- `cv2.hconcat([rgb_image, depth_vis])`: Horizontally concatenates the RGB image and the depth pseudo-color map into a dual-frame image.
- `cv2.imshow(...)` / `cv2.waitKey(1)`: Displays the concatenated depth estimation image; `waitKey(1)` refreshes the window.

```
class DEPTH_ESTIMATE(Node):
    def __init__(self,name):
        super().__init__(name)
        self.declare_parameter('init_pose', [90, 180, 0, 0, 90, 30])
        self.init_pose =
self.get_parameter('init_pose').get_parameter_value().integer_array_value
        self.declare_parameter('model_path','')
        self.model_path =
self.get_parameter('model_path').get_parameter_value().string_value
        self.declare_parameter('depth_colormap', 'JET')
        self.depth_colormap =
self.get_parameter('depth_colormap').get_parameter_value().string_value
        self.set_arm_client = self.create_client(ArmPosition, 'set_arm_joints')
        self.rgb_bridge = CvBridge()
        self.sub_rgb =
self.create_subscription(Image,'camera_rgb_topic',self.get_RGBImageCallBack,1)
        self.prev_time = time.time()
        self.frame_count = 0
        self.fps = 0.0

        self.colormap_dict = {
            'JET': cv2.COLORMAP_JET,
            'TURBO': cv2.COLORMAP_TURBO,
            'INFERNO': cv2.COLORMAP_INFERNO,
            'MAGMA': cv2.COLORMAP_MAGMA,
        }
        self.colormap = self.colormap_dict.get(self.depth_colormap.upper(),
cv2.COLORMAP_JET)

        try:
            self.depth_model = YOLO(self.model_path, task='depth')
        except Exception:
            self.depth_model = YOLO(self.model_path)
        self.get_logger().info(Fore.GREEN + 'Depth Estimate Model Loaded.' +
Fore.RESET)
        self._init_arm_pose()

    def _init_arm_pose(self):
        if not self.set_arm_client.wait_for_service(timeout_sec=10.0):
            self.get_logger().warn("Failed to connect to set_arm_joints server,
skipping init arm pose")
            return
        req = ArmPosition.Request()
        req.request_joints = list(self.init_pose)
        future = self.set_arm_client.call_async(req)
        future.add_done_callback(self._init_arm_done_callback)

    def _init_arm_done_callback(self, future):
        result = future.result()
```

```
        if not result.success:
            self.get_logger().warn(f"Init arm pose failed:
{list(result.result_joints)}, msg: {result.message}")

    def _draw_info(self, frame, infer_ms):
        cv2.putText(frame,f"FPS: {self.fps:.2f}",(10,
25),cv2.FONT_HERSHEY_SIMPLEX,0.6,(0, 255, 0),1,)
        cv2.putText(frame, f"Infer: {infer_ms:.1f} ms", (10,
45),cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 255), 1)

    def get_RGBImageCallBack(self,rgb_msg):
        rgb_image = self.rgb_bridge.imgmsg_to_cv2(rgb_msg,
desired_encoding='bgr8')
        infer_start = time.time()
        depth_map = None
        results = self.depth_model.predict(rgb_image, stream=True, verbose=False)
        for res in results:
            depth_map = getattr(res, 'depth', None)
        infer_end = time.time()
        if depth_map is None:
            self.get_logger().warn('No depth result in model output.', once=True)
            return

        if hasattr(depth_map, 'data'):
            depth_map = depth_map.data
        if hasattr(depth_map, 'cpu'):
            depth_map = depth_map.cpu().numpy()
        depth_map = np.squeeze(depth_map)
        depth_vis = cv2.normalize(depth_map, None, 0, 255,
cv2.NORM_MINMAX).astype(np.uint8)
        depth_vis = cv2.applyColorMap(depth_vis, self.colormap)
        if depth_vis.shape[:2] != rgb_image.shape[:2]:
            depth_vis = cv2.resize(depth_vis, (rgb_image.shape[1],
rgb_image.shape[0]))

        self.frame_count += 1
        curr_time = time.time()
        time_diff = curr_time - self.prev_time
        if time_diff >= 1.0:
            self.fps = self.frame_count / time_diff
            self.frame_count = 0
            self.prev_time = curr_time
        infer_time = (infer_end - infer_start) * 1000
        self._draw_info(rgb_image, infer_time)
        self._draw_info(depth_vis, infer_time)
        cv2.putText(rgb_image, "RGB", (10, rgb_image.shape[0] -
15),cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 0), 1)
        cv2.putText(depth_vis, "Depth", (10, depth_vis.shape[0] -
15),cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 0), 1)

        combined_frame = cv2.hconcat([rgb_image, depth_vis])
        cv2.imshow("Depth Estimate Panel", combined_frame)
        cv2.waitKey(1)
```